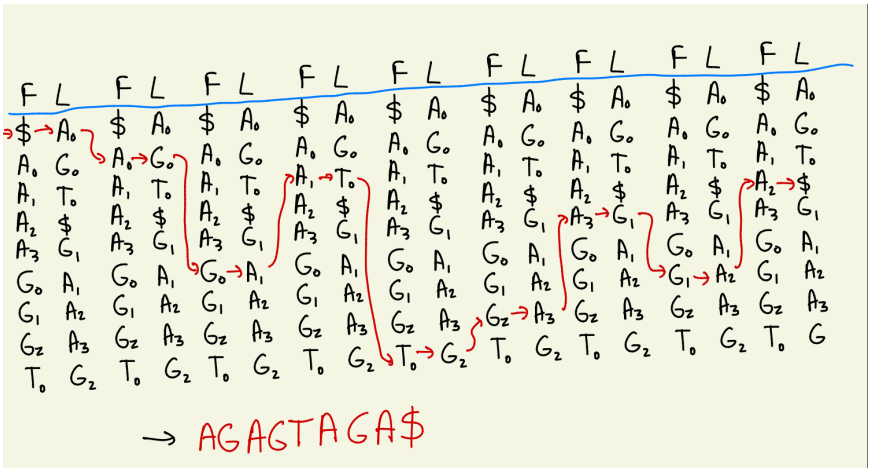
Ashrith Kollu

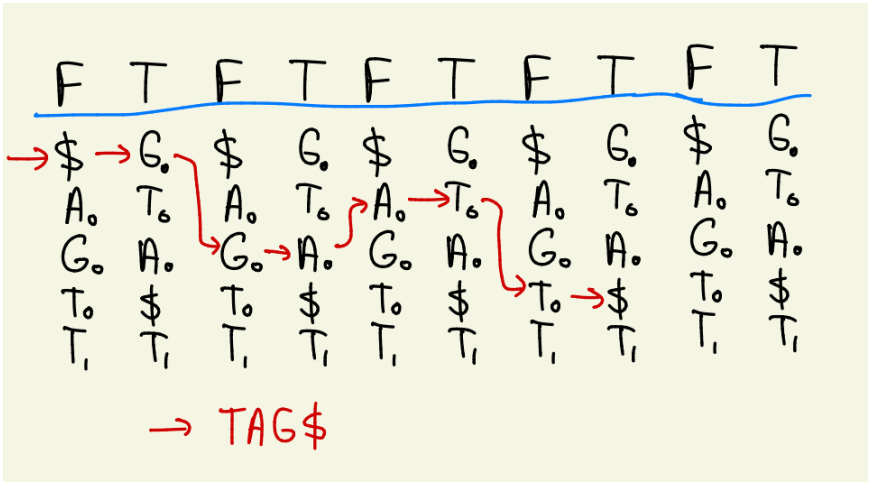
akollu1

1. a) $CATCAT, T$CATCA, AT$CATC, CAT$CAT, TCAT$CA, ATCAT$C, CATCAT$ need to be sorted –> $CATCAT, AT$CATC, ATCATC, CATCAT, CATCAT$, T$CATCA, TCAT$CA. Taking the last column of each gives us TCCT$AA. Thus, BWT(T) = TCCT$AA.

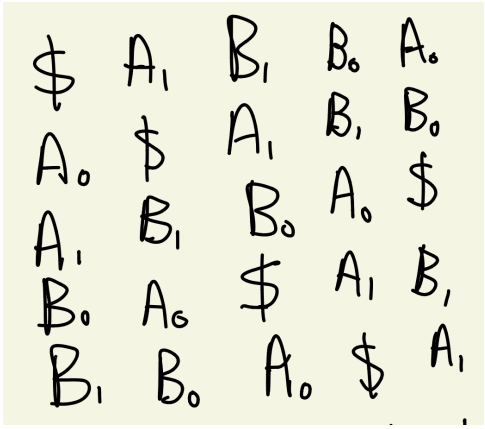
b)



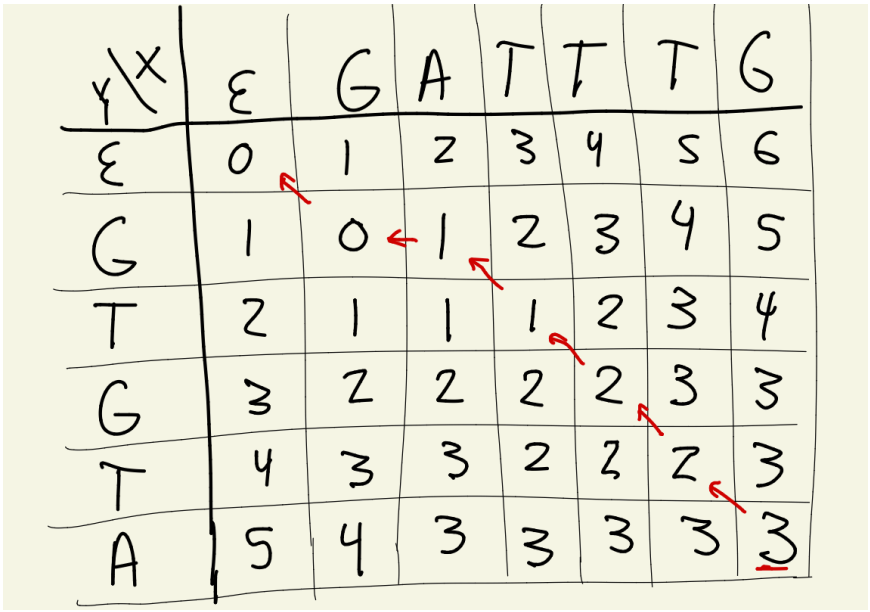
c)



With string GTA$T, we get an early termination of the BWT reversal before all the letters are visited. The reversal ends at only TAG$. Since the BWT of a string is a special permutation that allows for the reconstruction of the full original string via reversal, the fact that the reversal failed on this string indicates that it is not a valid BWT. There is no way to visit all the characters in the BWT to reconstruct a string with the same number of characters. Since this is a characteristic property of BWTs, this cannot be a BWT for any string.

2. 

T is ABBA$ and BWT(T) is AB$BA. Since we always have $ unique and column 1 can be found by putting all the characters in alphabetical order, we will always have information about the letter 2 positions before the $ and 2 positions after given column 3. This gives us 3 of the 5 necessary characters in the string, `_BB_$` and since the other two have to be A's, we know the string is ABBA$. Then finding the BWT of that is trivial by filling in the rest of the table and extracting the last column. Even though it worked out in this case, it is not always possible to reconstruct the string from a random row that isn't the last one because we don't have the property of preserved rank order that ensures we always know which letter and rank go in each position.

3. a) 

Y \ X	ε	G	A	T	T	T	G
ε	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
T	2	1	1	1	2	3	4
G	3	2	2	2	2	3	3
T	4	3	3	2	2	2	3
A	5	4	3	3	3	3	3

Edit distance = 3

b)

```
Alignment:

G  A  T  T  T  G
|     |     |
G  -  T  G  T  A
```

There are 2 substitutions and 1 gap in the alignment.

4. a) Since X and R(X) differ by just one rotation, we will always be able to align X with R(X) by introducing a gap in the beginning with X[0] and at the end with R(X)[n]. The rest of the sequence is sure to match, giving us a maximum edit distance of 2 regardless of what X is.

```
X   : X[0]  X[1]  X[2]  ...  X[n-1]  X[n]   -
              |     |     |     |      |
R(X):   -   X[1]  X[2]  ...  X[n-1]  X[n]  X[0]
```

b) X' = ACGTACGT This would make R(X') = TACGTACG, which mismatches at every single position to yield a Hamming distance of 8.

c) The edit distance between X' and R(X') is exactly 2, as stated as the max edit distance from part a. It can't be smaller because with a hamming distance of 8, using only mismatches to get the strings to align would yield 8 edits. This type of string where the rotation mismatches more bases than twice the number of positions rotated requires gaps to be introduced to get the smallest edit distance. With a rotation of only one position, two gaps are required to always get a match back. This accounts for the missing base at the first position of R(X') and the extra base (that used to be in the first position) at the end of R(X'). Since the strings are the same length and hamming distance is greater than 2 * positions rotated = 2, we need two gaps. One shifts the alignment to offset for the rotation while the other accounts for the overhang at the end that is introduced from shifting the sequence when both are the same length. Thus, this number cannot be any smaller.